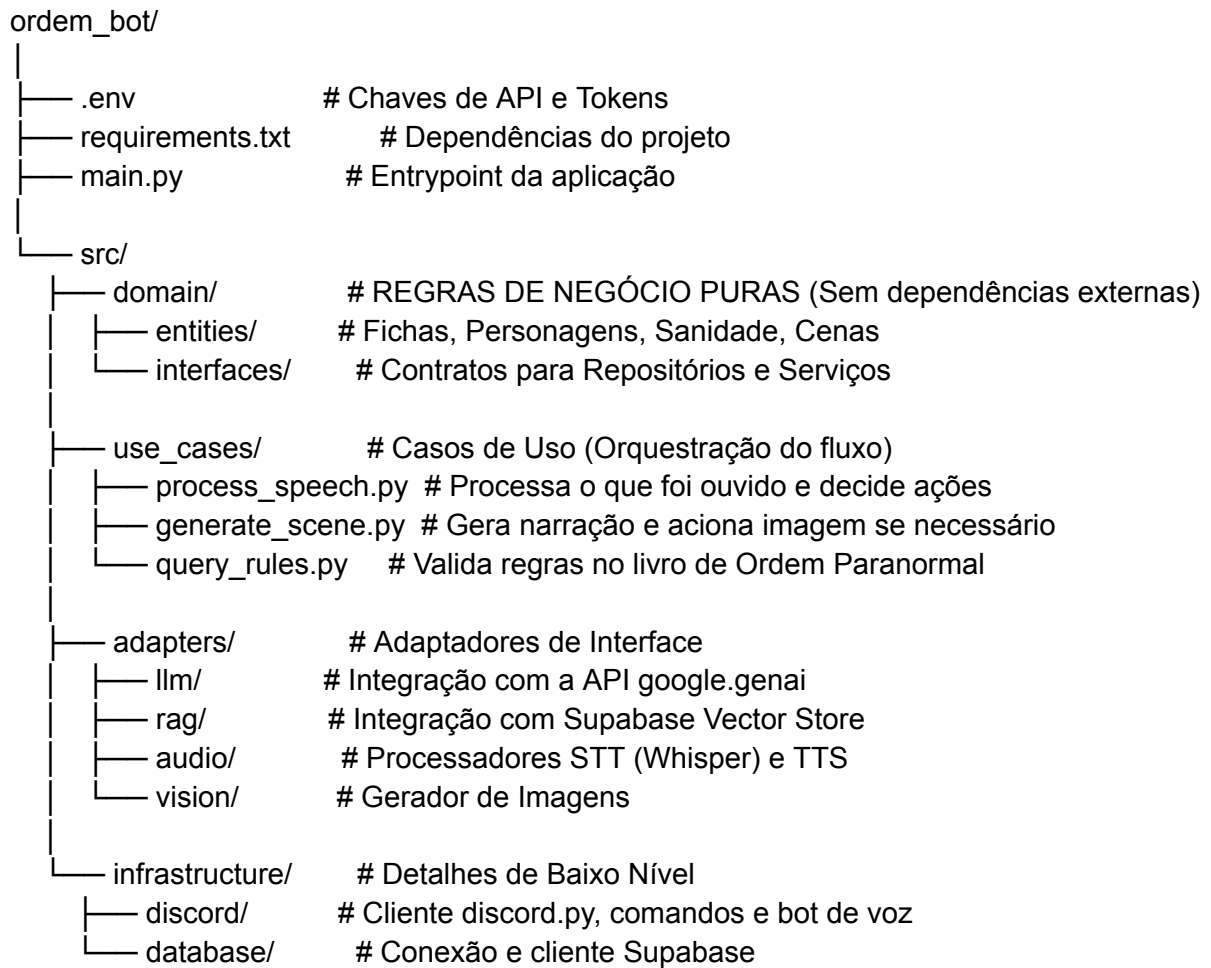


Isso e uma ideia geral e esta sujeita a alteracoes

1- possivel arquitetura



2- stack

Python

Discord

Gemini

Supabase

Whisper

Clean

3- fases de implementacao

Fase 1: Setup do Ambiente e Estrutura Base

- **Ações:**

1. Criar o ambiente virtual Python (`python -m venv venv`).
2. Instalar dependências essenciais: `discord.py`, `PyNaCl` (para áudio no Discord), `google.genai`, `supabase`, `python-dotenv` e `pydantic`.
3. Criar o arquivo `.env` para centralizar as chaves de API:

- `DISCORD_TOKEN`
- `GEMINI_API_KEY`
- `SUPABASE_URL / SUPABASE_KEY`
- `TTS_STT_API_KEYS`

4. Subir o esqueleto de pastas e implementar a conexão básica no `main.py`.

Fase 2: Motor de Regras e RAG (Supabase + Vector Store)

Para garantir que o bot entenda exclusivamente *Ordem Paranormal* (testes de perícia, presença perturbadora, custos de PE, sanidade):

- **Ações:**
 1. **Extração e Chunking:** Processar os PDFs das regras de *Ordem Paranormal* dividindo-os em blocos de texto significativos.
 2. **Vetorização:** Gerar os *embeddings* desses blocos e salvá-los em uma tabela com a extensão `pgvector` ativa no Supabase.
 3. **Adapter de Busca:** Criar o serviço RAG que, a cada dúvida de mecânica ("como funciona o teste de sanidade contra um Zumbi de Sangue?"), busca os 3 trechos mais relevantes no Supabase para injetar no contexto da IA.

Fase 3: Cérebro Cognitivo e Sistema de Memória

A narrativa não pode alucinar nem exceder o limite de contexto.

- **Ações:**
 1. **Gerenciador de Prompts:** Criar o prompt do sistema com a personalidade do Mestre (tom sombrio, misterioso e imparcial).
 2. **Memória de Curto Prazo:** Manter um buffer contendo apenas os últimos `$N$` turnos de fala da sessão.
 3. **Trabalhador de Resumo (Memória de Longo Prazo):** A cada `$X$` turnos, disparar uma chamada assíncrona usando a biblioteca `google.genai` para sintetizar os acontecimentos em *PONTOS DE INSIGHT* (ex: "Os agentes encontraram uma chave enferrujada no porão").
 4. **Injeção de Contexto:** A chamada principal do bot sempre receberá:
`$$\text{Prompt do Mestre} + \text{Memória de Longo Prazo} + \text{Contexto das Regras (RAG)} + \text{Últimas Falas}$$`

Fase 4: Pipeline de Áudio em Tempo Real

O ciclo de escuta e resposta em call.

- **Ações:**
 1. **Audio Sink (Ouvir):** Configurar o leitor de voz do `discord.py` para capturar os pacotes de áudio PCM dos membros no canal.
 2. **Speech-to-Text (STT):** Enviar o áudio capturado para a API de transcrição e converter em texto.
 3. **Text-to-Speech (TTS):** Receber a resposta gerada pelo cérebro da IA e converter em um fluxo de áudio com voz sombria.
 4. **Audio Player (Falar):** Transmitir o arquivo de áudio resultante de volta para o canal de voz do Discord.

Fase 5: Módulo Visual e Ilustrações

Geração de imagens de locais ou ameaças sem bloquear a narração.

- **Ações:**
 1. **Estruturação da Resposta:** Configurar a IA para retornar uma estrutura (como JSON) contendo dois campos:
 - **fala_narracao:** O texto que será lido na call.
 - **prompt_imagem** (Opcional): Descrição visual detalhada caso a cena exija uma imagem (ex: "Um corredor escuro coberto de lodo vermelho").
 2. **Geração Assíncrona:** Se **prompt_imagem** estiver presente, disparar em segundo plano o pedido para a API de geração visual.
 3. **Envio no Chat:** Postar a imagem gerada no canal de texto do servidor enquanto o bot fala o áudio na call.

Fase 6: Validação e Testes de Integração

- **Ações:**
 1. **Testes Unitários:** Validar as funções de cálculo de dados e regras do sistema no **domain/** via **pytest**.
 2. **Teste de Carga na Call:** Testar o tempo de resposta entre a fala do jogador e a resposta do bot (latência do STT $\rightarrow$ LLM $\rightarrow$ TTS).
 3. **Ajuste de Sensibilidade de Voz:** Configurar o limite do *Audio Sink* para ignorar ruídos de fundo dos microfones.